



Universidade do Minho

Escola de Engenharia

Programação **I**mperativa

LETI :: 2022/23

Outubro 2022

© António Esteves

© Slides adaptados de:

CSC230 - C e Software Tools, Computer Science Faculty, Duke University, USA

BE5B99CPL – C Programming Language, Jan Faigl, Czech Technical University in Prague

Bibliografia essencial:

C Programming: A Modern Approach, capítulo 7

4. Tipos de Dados Fundamentais do C

➤ Tipos de dados nativos

- inteiros
- reais
- caractere
- `_Bool`

➤ Conversões de tipo

- explícitas
- implícitas

➤ Exercícios

- Uma **variável** = **tamanho** em memória + sua **interpretação**
- A **interpretação** do valor representado por uma variável depende de ser um
 1. **inteiro** (com ou sem sinal)
 2. **real** (precisão simples ou precisão dupla)
 3. **caratere** (código ASCII ou valor inteiro ou ...)

- Em C as variáveis têm um **tipo** definido **estaticamente**
 - o tipo tem que ser especificado quando a variável é criada e **não** pode **mudar** depois
- As linguagens com atribuição **dinâmica** do **tipo** permitem alterar o tipo das variáveis durante o seu tempo de vida, e por isso são mais flexíveis
 - exemplos: Python, Perl, Ruby, JavaScript

- O **tipo** de uma **variável** define a sua **interpretação**
- Exemplo: suponhamos que temos o seguinte valor de 32 bits armazenado em memória, a partir do endereço **addr**

01000100|01000011|01000010|01000001

addr+3

addr+2

addr+1

addr

- se o tipo do valor for **float**, é interpretado como sendo o número real **781,03521728515625**
- se o tipo do valor for **unsigned int**, é interpretado como sendo o número inteiro **1145258561**
- se o tipo do valor for **char**, é interpretado como sendo a sequência de 4 caracteres ASCII **ABCD**

Tipos de dados nativos (1)

- Os tipos nativos do C são os inteiros e os reais em vírgula flutuante

O tipo de dados **lógico** foi introduzido na norma **C99**
- As palavras-chave que identificam os tipos de dados nativos do C são:
 - Tipos inteiros: **long**, **int**, **short** e **char**

Modificadores da gama de representação: **signed** e **unsigned**
 - Tipos em vírgula flutuante: **float**, **double** e **long double**
 - Tipo para caracteres: **char**
 - Tipo de dados associado a um conjunto sem valores possíveis: **void**
 - Tipo de dados booleano: **_Bool**
- Além destes tipos, podemos **definir novos tipos** de dados utilizando a palavra-chave **typedef**

- O tamanho ocupado em memória pode ser obtido com o operador **sizeof()**, utilizando como argumento o **nome do tipo** de dados ou o **nome da variável** do(a) qual pretendemos saber o tamanho
- Exemplos:

```
int i;
```

```
printf("tamanho do int: %lu\n", sizeof(int));
```

```
printf("tamanho de i: %lu\n", sizeof(i));
```

04-tipos.c

➤ Estes tipos são compostos a partir dos tipos primários

- **vetor** (*array*)
- **função**
- **apontador**
- **struct**
- **union**

Discutiremos estes tipos mais tarde

➤ Tipos **enumerados**

Discutiremos os enumerados mais tarde

➤ Números **complexos**

Não utilizaremos nesta UC

➤ Tipos de dados nativos

- **inteiros**
- reais
- caratere
- `_Bool`

➤ Conversões de tipo

- explícitas
- implícitas

➤ Exercícios

Tipos de dados inteiros

- O **tamanho** dos tipos de dados inteiros **não é definido pela norma** do C, mas sim pela **implementação** (compilador e bibliotecas)

Os tamanhos podem diferir entre implementações, especialmente entre dispositivos computacionais de 16-bits e 64-bits

- A norma do C apenas especifica uma **relação entre os tamanhos** dos tipos inteiros:
 - **short** $\leq$ **int** $\leq$ **long**
 - **unsigned short** $\leq$ **unsigned int** $\leq$ **unsigned long**
- O tipo de dados fundamental que é o **int** ocupa normalmente **4 bytes** em arquiteturas de 32 e de 64 bits
- Valor **mínimo** e **máximo** para alguns tipos inteiros:

Tipo	Valor mínimo	Valor máximo
short	-32768	32767
int	-2147483648	2147483647
unsigned int	0	4294967295

Tipos de dados inteiros com e sem sinal



- Além do número de bytes ocupados por cada tipo inteiro, podemos ainda distinguir:

- **signed** (por omissão)
- **unsigned**

Uma variável de um tipo unsigned não pode representar um número negativo

- Exemplo dos tipos de inteiros que ocupam 1 byte:

unsigned char: valores de 0 a 255

signed char: valores de -128 a 127

```
unsigned char uc = 127;
```

```
char          sc = 127;
```

```
printf("Valor inicial de uc=%i e de sc=%i\n", uc, sc);
```

```
uc = uc + 2;
```

```
sc = sc + 2;
```

```
printf("Novo valor de uc=%i e de sc=%i\n", uc, sc);
```

Valor inicial de uc=127 e de sc=127

Novo valor de uc=129 e de sc=-127

04-signed-unsigned-char.c

Valores mínimo e máximo dos inteiros (1)

- O **tamanho** (em bits) e os valores máximo e mínimo dos inteiros **dependem da plataforma de execução**
- Os valores são definidos num ficheiro *header*

`/usr/include/limits.h` (em Linux)

`...\\mingw64\\lib\\gcc\\x86_64-w64-mingw32\\8.1.0\\include-fixed\\limits.h` (em Windows e com gcc do pacote Mingw-w64 versão 8.1.0)

Tipo	#bits	Valor
Mínimo de qualquer tipo sem sinal (<i>unsigned</i>)	---	0
Mínimo de char	8	-128
Máximo de char	8	127
Máximo de unsigned char	8	255
Mínimo de short	16	-32 768
Máximo de short	16	32 767
Máximo de unsigned short	16	65 535

Valores mínimo e máximo dos inteiros (2)

Tipo	#bits	Valor
Mínimo de int	32	-2 147 483 648
Máximo de int	32	2 147 483 647
Máximo de unsigned int	32	4 294 967 295
Mínimo Máximo de long	32	(o mesmo que int)
Máximo de unsigned long	32	(o mesmo que unsigned int)
Mínimo de long long	64	-9 223 372 036 854 775 808
Máximo de long long	64	9 223 372 036 854 775 807
Máximo de unsigned long long	64	18 446 744 073 709 551 615

➤ Tipos de dados nativos

- inteiros
- **reais**
- caractere
- _Bool

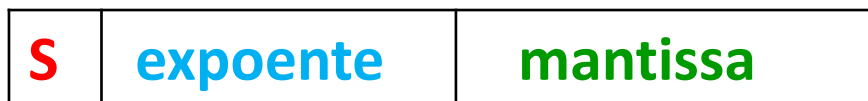
➤ Conversões de tipo

- explícitas
- implícitas

➤ Exercícios

Tipos de dados reais em vírgula flutuante (1)

- O C disponibiliza 3 tipos de dados em vírgula flutuante:
 - **float** - vírgula flutuante de **precisão simples**
 - **double** - vírgula flutuante de **precisão dupla**
 - **long double** - vírgula flutuante de **precisão estendida**
- Quem define a precisão dos tipos não é o C, mas a norma IEEE 754
 - Um número real **x** armazenado como:



representa o valor decimal:

$$x = (-1)^S * 1, mantissa * 2^{expoente - excesso}$$

- O **excesso** ($2^{\#bitsExpoente}-1$) permite armazenar o expoente sempre como um número positivo

- O tamanho do **expoente** (em bits) determina a **gama** de números que podem ser representados
- O tamanho da **mantissa** (em bits) determina a **precisão** dos números que podem ser representados

➤ Norma IEEE de vírgula flutuante e **precisão simples**:

- 1 bit para o **sinal** (0='+' e 1='-')
- 8 bits para o **expoente** (em excesso de 127),
256 valores
- 23-bits (+ 1 bit escondido) para a **mantissa**
- 6 dígitos decimais de precisão

32 bits

➤ **precisão dupla**:

- 1 bit para o **sinal**
- 11 bits para o **expoente** (em excesso de 1023),
2048 valores
- 52-bits (+ 1 bit escondido) para a **mantissa**,
 $\approx 4,5 \times 10^{15}$ valores
- **15** dígitos decimais de precisão

64 bits

➤ Menor valor positivo normalizado

- precisão simples (**float**): $2^{-126} \approx 10^{-38}$
- precisão dupla (**double**): $2^{-1022} \approx 10^{-308}$
- **Q:** O mínimo dum **float** será suficientemente pequeno para guardar o diâmetro de um átomo (em metros)?

➤ Maior valor positivo normalizado

- precisão simples (**float**): $2^{127} \approx 10^{38}$
- precisão dupla (**double**): $2^{1023} \approx 10^{308}$
- **Q:** O máximo do **double** será suficientemente grande para guardar o número de átomos no universo? Ou a distância até ao limiar observável do universo, usando como unidade o diâmetro do átomo?

➤ **long double** = 128 bits

permite mais precisão que o **double** mas a gama de valores é a mesma (mesmo nº de bits do expoente)

➤ Tipos de dados nativos

- inteiros
- reais
- **caratere**
- `_Bool`

➤ Conversões de tipo

- explícitas
- implícitas

➤ Exercícios

Carateres – o tipo **char**

- Para guardar um caratere usa-se o tipo de dados **char**
- O tipo **char** é um **inteiro** que ocupa 1 byte
- O valor usado para representar cada um dos carateres (letras, algarismos, espaços em branco, símbolos gráficos) segue a codificação **ASCII**

ASCII - American Standard Code for Information Interchange

- Um literal (ou constante) do tipo **char** pode ser escrito incluindo um caratere entre plicas: *exemplo 'a'*

```
char c = 'a' ;
```

```
printf("O valor de c como int e' %i e como caratere e' '%c' \n", c, c) ;
```

04-char.c

```
gcc 04-char.c -o pchar && ./pchar
```

```
O valor de c como int e' 97 e como caratere e' 'a'
```

- Existem vários **carateres de controlo** definidos para interagir com dispositivos de saída (como o terminal do SO ou uma impressora)
 - `\t` – um avanço na horizontal, `\n` – mudança de linha, `\a` – sinal sonoro,
 - `\b` – apagar o caratere anterior, `\r` – ir para o início da linha,
 - `\f` – mudança *de página*, `\v` – um avanço/espço na vertical

- A codificação ASCII é específica para caracteres ocidentais: pontuação, dígitos, letras maiúsculas e minúsculas
- Apenas os **primeiros 128 valores** (0-127 em decimal) são normalizados
- A interpretação dos **restantes 128 valores** (128-255 em decimal) não é normalizada

ASCII – códigos normalizados (0-127)



Universidade do Minho
Escola de Engenharia

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174	|	
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177		DEL

ASCII: "uma" interpretação dos códigos 128-255

128	Ç	144	É	161	í	177	░	193	⊥	209	〒	225	β	241	±
129	ü	145	æ	162	ó	178	▒	194	⌞	210	π	226	Γ	242	≥
130	é	146	Æ	163	ú	179		195	⌟	211	⌌	227	π	243	≤
131	â	147	ô	164	ñ	180	⌞	196	—	212	⌞	228	Σ	244	∫
132	ä	148	ö	165	Ñ	181	⌞	197	⊕	213	ƒ	229	σ	245	∫
133	à	149	ò	166	²	182	⌞	198	⌞	214	π	230	μ	246	÷
134	â	150	û	167	°	183	π	199	⌞	215	⌞	231	τ	247	≈
135	ç	151	ù	168	¿	184	⌞	200	⌞	216	⌞	232	Φ	248	°
136	ê	152	—	169	—	185	⌞	201	⌞	217	⌞	233	⊕	249	.
137	ë	153	Ö	170	¬	186	⌞	202	⌞	218	⌞	234	Ω	250	.
138	è	154	Û	171	½	187	⌞	203	〒	219	■	235	δ	251	√
139	ï	156	£	172	¼	188	⌞	204	⌞	220	■	236	∞	252	—
140	î	157	¥	173	¡	189	⌞	205	=	221	■	237	φ	253	²
141	ï	158	—	174	«	190	⌞	206	⌞	222	■	238	ε	254	■
142	Ä	159	f	175	»	191	⌞	207	⌞	223	■	239	∩	255	
143	Å	160	á	176	░	192	⌞	208	⌞	224	α	240	≡		

Source: www.LookupTables.com

➤ Tipos de dados nativos

- inteiros
- reais
- caractere
- **_Bool**

➤ Conversões de tipo

- explícitas
- implícitas

➤ Exercícios

Tipo booleano: `_Bool`

- O **C99** introduziu o tipo de dados booleano `_Bool`
- `_Bool` `var_booleana;`
- O valor **true** é qualquer valor inteiro diferente de zero
- O valor **false** é zero
- Os valores **true** e **false** são definidos, juntamente com o tipo **bool**, no ficheiro *header* `stdbool.h`

```
#define false 0
#define true 1
#define bool _Bool // bool é um nome alternativo
```

...

- Nas normas do C anteriores ao C99, o tipo de dados booleano não estava pré-definido
- Uma definição equivalente à que é feita em `stdbool.h` seria usar um tipo enumerado com nome `_Bool` com 2 etiquetas:

```
#define FALSE 0
#define TRUE 1
```

Questão

- Suponho que tem o número inteiro **107** guardado numa variável **short int vs**, a qual ocupa **2 bytes** em memória.
- Qual o valor binário/hexadecimal guardado em cada um dos 2 bytes de **vs**?

| | <-- byte mais significativo

| | <-- byte menos significativo

- Suponha agora que tem a *string* "**107**" guardada como uma sequência de 3 **char**'s em memória.
- Qual o valor binário/hexadecimal guardado em cada um dos 3 bytes ocupados pela sequência?

| | | <-- | 1º char | 2º char | 3º char |

➤ Tipos de dados nativos

- inteiros
- reais
- caractere
- `_Bool`

➤ **Conversões de tipo**

- explícitas
- implícitas

➤ Exercícios

Conversões de tipo (1)

- Uma conversão de tipo transforma um valor do seu tipo de dados original para um valor noutra tipo de dados diferente
- A conversão de tipo pode ser:
 - **Implícita** – **efetuada automaticamente pelo compilador**, por exemplo para que um valor seja do tipo da variável a que ele é atribuído
 - **Explícita** – **escrita colocando o operador de cast** antes do valor a converter → **(novoTipo) valor**
- A conversão de **int** para **double** é implícita

Um valor do tipo `int` pode ser usado numa expressão, onde se espera um valor do tipo `double`. O valor `int` é automaticamente convertido para `double`

- **Exemplo:**

```
double x;
```

```
int     i=1;
```

```
x = i; // o valor 1 do tipo int é convertido
```

```
// automaticamente para o valor 1.0 do tipo double
```

- **As conversões de tipo implícitas são seguras**

➤ Exemplo:

```
unsigned char uc;  
int          i;  
float        f;  
double       d;  
...
```

Explícita

```
f = (float) i;
```

Implícitas

```
d = uc + (i * f);
```

Conversões de tipo explícitas

- As conversões de valores **double** para **int** têm que ser **explicitamente** especificadas com o **operador de cast**
- Nestas conversões a parte fracionária será **truncada**

Exemplo:

```
double x = 1.75; // declaração de x como variável double
int     i;       // declaração de i como variável int
i = (int)x;      // o valor 1.75 (do tipo double) é
                 // truncado para 1 (do tipo int)
```

- As conversões de tipo explícitas podem gerar resultados errados e imprevistos nos programas

Exemplos:

```
double d = 1e30;
int     i = (int)d;

// i é -2147483648
// que é  $\sim -2 \cdot 10^9 \neq 10^{30}$ 
// 1030 não cabe num int)
```

```
long long llv = 50000000000L;
int     i     = (int)llv;

// i é 705032704
// (llv é truncado a 4 bytes)
```

04-conversao-tipo-1.c

Conversão de valores com sinal para sem sinal

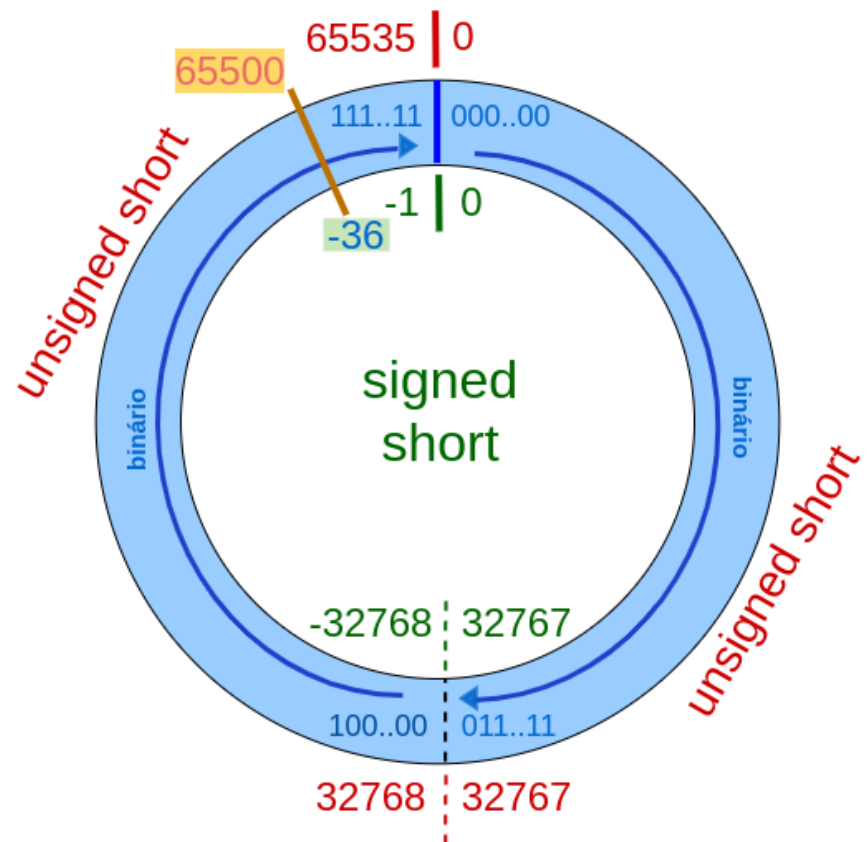
- Esta conversão só faz sentido se tivermos a **certeza** que o valor **signed** a converter para **unsigned** é **positivo**

```
short          a;  
unsigned short b;  
a = -36;  
b = (unsigned) a;  
a = (signed) b;
```

Resultado:

```
b = 65500 // errado  
a = -36   // OK
```

04-conversao-tipo-2.c



Conversão de valores sem sinal para com sinal

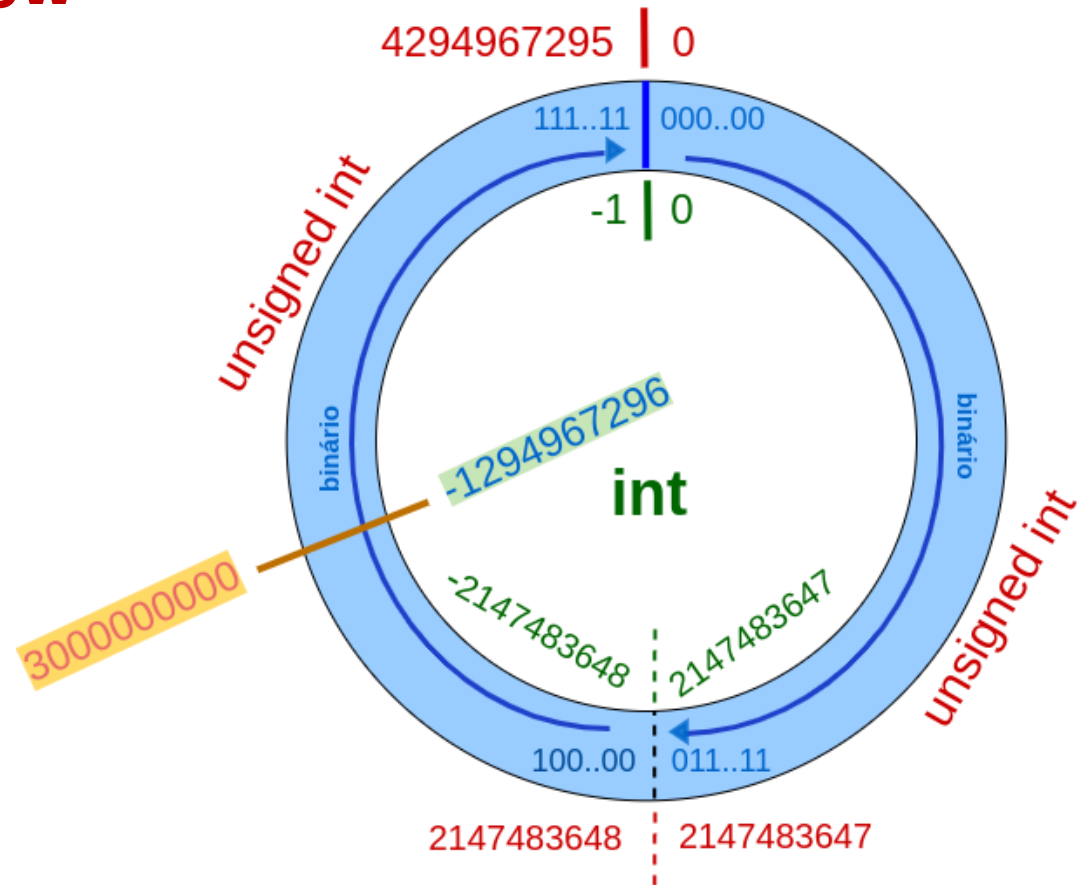
- Se o valor sem sinal for superior ao máximo permitido pelo tipo com sinal naturalmente o resultado final será incorreto porque não cabe na gama de representação
→ problema de **overflow**

```
int a;  
unsigned int b;  
  
b = 3000000000;  
a = (int) b;
```

Resultado:

b = 3000000000
a = -1294967296

04-conversao-tipo-3. c



Conversão de vírgula flutuante para inteiro

- Ao converter um **real para inteiro**, o número real é **truncado** e como resultado ficamos apenas com a **parte inteira**, a parte fracionária é descartada
 - $+3.999 \rightarrow 3$
 - $-3.999 \rightarrow -3$
- Naturalmente, ao descartar a parte fracionária haverá **redução de precisão**
- Se a **parte inteira** do real for **superior ao máximo do tipo inteiro** de destino, ocorrerá **overflow**

```
float f = 1.0e10;  
int i;  
i = f;
```

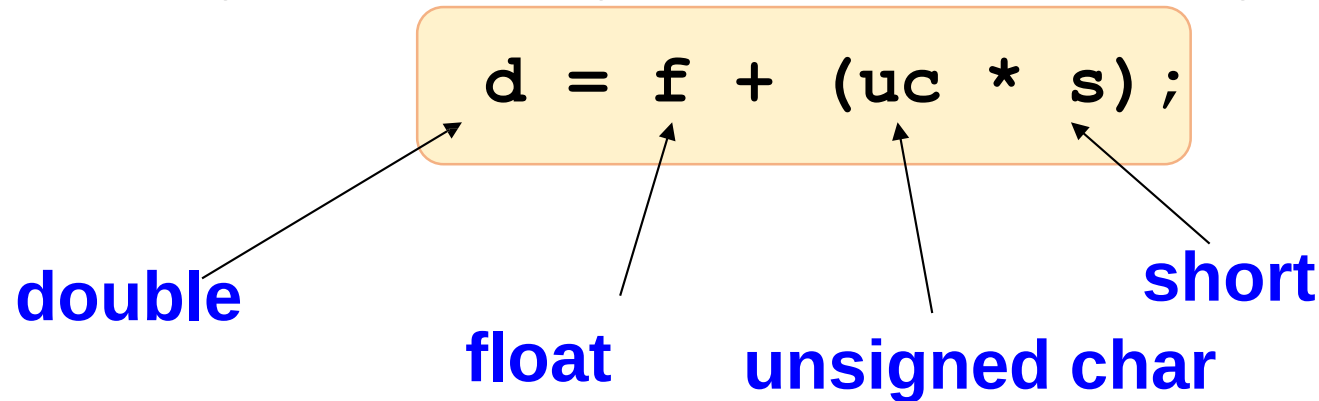
04-conversao-tipo-4.c

Resultado:

```
f = 10000000000.0  
i = -2147483648
```

- Inteiro → vírgula flutuante
 - quando o valor inteiro não puder ser representado exatamente em vírgula flutuante, a conversão é feita para o valor **mais próximo** (maior ou menor) que é representável em vírgula flutuante
- Precisão dupla → precisão simples
 - se o valor não puder ser representado exatamente, a conversão é feita para o valor **mais próximo** (superior ou inferior)
 - Como a precisão dupla dispõe de mais bits para o expoente e para a mantissa do que a precisão simples, pode ocorrer **overflow** (valor demasiado grande, substituído por $+\infty$) ou **underflow** (valor demasiado pequeno, substituído por zero)

- Em expressões quem misturam tipos diferentes



- o compilador **converte os valores para um tipo comum** antes de avaliar a expressão
- Os **char's** e **short's** são sempre convertidos para **int** (ou **unsigned int**) antes de avaliar as expressões

- Ao combinar numa expressão valores inteiros e reais, o compilador converte os inteiros para vírgula flutuante
- Ao combinar numa expressão **float's** e **double's**, o compilador converte os **float's** para **double's**
- A ordem das conversões importa, como se pode ver na próxima tabela

Conversões de tipo implícitas (3)

Regra	Se um dos operandos é ...	e o outro operando é ...	então converte o outro operando para ...	senão ...
#1	<code>long double</code>	qualquer tipo	<code>long double</code>	#2
#2	<code>double</code>	qualquer tipo	<code>double</code>	#3
#3	<code>float</code>	qualquer tipo	<code>float</code>	#4
#4	<code>unsigned long int</code>	qualquer tipo	<code>unsigned long int</code>	#5
#5	<code>long int</code>	<code>unsigned int</code>	<code>unsigned long int</code>	#6
#6	<code>long int</code>	qualquer tipo	<code>long int</code>	#7
#7	<code>unsigned int</code>	qualquer tipo	<code>unsigned int</code>	#8
#8			(se ambos os operandos são <code>int</code> , não há conversões)	

Conversões de tipo implícitas: exemplo

double

unsigned char

float

short int

```
d = f + (uc * s) ;
```

- Antes de avaliar a expressão:
→ converter **uc** para **unsigned int** e **s** para **int**
- Antes de multiplicar:
→ converter **s** para **unsigned int** (regra #7)
- Antes de somar:
→ converter o resultado da multiplicação para **float** (regra #3)
- Antes da atribuição a **d**:
→ converter o resultado da soma para **double** (regra #2)

- Converter o código **ASCII** de um algarismo **para um inteiro**:

```
unsigned char c, c2;  
c = (unsigned char) getchar();  
unsigned int n;  
n = c - '0';  
printf("' %c' -> %d\n", c, n);
```

48	30	060	0	0
49	31	061	1	1
50	32	062	2	2
51	33	063	3	3
52	34	064	4	4
53	35	065	5	5
54	36	066	6	6
55	37	067	7	7
56	38	070	8	8
57	39	071	9	9

- Converter um **inteiro 0..9 para ASCII**:

```
c2 = (char) (n + '0');
```

04-conversao-int-ascii.c

- Como convertemos uma *string* ASCII (por ex. "**12**") para um inteiro? E um inteiro para a *string* ASCII correspondente?

➤ Tipos de dados nativos

- inteiros
- reais
- caractere
- `_Bool`

➤ Conversões de tipo

- explícitas
- implícitas

➤ Exercícios

Exercício 1: Para cada uma das constantes na tabela a seguir indique (i) se a constante é válida ou inválida e (ii) qual o tipo da constante (se for válida) ou porque é inválida

Constante	Válida? (sim/não)	Explicação (tipo <u>OU</u> porque é inválida)
486	Sim	int (em decimal)
98.6	Sim	double
98.6f	Sim	float (o f indica que é precisão simples – float – e não double)
02.479	Não	numa constante octal não são permitidas casas decimais
0.2479	Sim	double
"A"	Sim	cadeia de caracteres (2 char's incluindo o terminador '\0')
'A'	Sim	um char (sem '\0')
'ABC'	Não	a plica só é permitida em caracteres individuais
'\n'	Sim	uma sequência de escape que representa o caratere mudança de linha
000042	Sim	octal ($4 * 8 + 2 = 34$ em decimal)
0ffH	Não	não existe 'f' em octal
0xC2F9	Sim	hexadecimal
+37.1	Sim	double
.0000	Sim	double
0xFLU	Sim	hexidecimal (valor=15, unsigned long)
0x48.6	Não	numa constante hexadecimal não são permitidas casas decimais
0x486e1	Sim	'e' é um dígito válido em hexidecimal
486e1	Sim	double ($486 * 10^1 = 4860$)
0486e1	Não	'e' não é um dígito válido em octal
0486	Não	'8' não é um dígito válido em octal

Exercício 2: tabela ASCII

- Elaborar um programa que escreve no terminal todos os caracteres cujo código ASCII está no intervalo 32-127
- Ajuda:
 - Escrever um ciclo que escreve no terminal os números inteiros no intervalo 32..127
 - Escrever uma instrução que usa o **printf** para mostrar um único caractere no terminal
 - Combinar os 2 passos anteriores

Exercício 3: Conversões

- Dadas as seguintes inicializações de variáveis:
 - `short` `a = -1;`
 - `int` `b = -2;`
 - `unsigned int` `c = 2147483648;`
- Indique o que seria escrito no terminal do SO se as variáveis onde se guardam os resultados das conversões seguintes fossem escritas com `printf`?
 - `unsigned short` `d = (unsigned short) a;`
 - `unsigned int` `e = (unsigned int) b;`
 - `short` `f = (short) d;`
 - `int` `g = (int) e;`
 - `short` `h = (short) a;`
 - `int` `i = (int) a;`
 - `int` `j = (int) c;`